

Kairos: A Compute-Efficient Hybrid MoE Diffusion Architecture for Multimodal Edge Foundation Models

A Proof-of-Concept Report

Fabien Furfaro

September 17, 2026

Abstract

Sparse Mixture-of-Experts (MoE) routing is usually sold as an almost-free lunch: keep the total parameter count high, activate only a small fraction per token, and pay only for what you use. This promise breaks down during *training* on memory-bandwidth-limited hardware: Kairos’s MoE backward pass is bandwidth-gated by the *total* parameter count, not the *active* one, a distinction FLOP-centric treatments of MoE efficiency omit. Kairos is a proof-of-concept multimodal model combining bidirectional gated DeltaNet linear attention with bidirectional sliding-window attention through shared projections (LiZAttention2, TPTT-inspired), a block-wise Attention-Residual aggregator, a sparse MoE feed-forward layer, and a masked-diffusion training objective over a shared byte-level latent space spanning text, image, audio, video, lidar and robot-control modalities. We give code-derived algorithms for the multi-scale codec and cross-session memory gate (for which we found no direct literature precedent), the hybrid attention block, the Block-AttnRes aggregator, and the masked-diffusion curriculum, a roofline analysis of a 200M-total/25M-active target design point ($\mathcal{I}_{\text{train}} = 1.5$ FLOP/byte against ridge points of 156–295 FLOP/byte on A100/H100-class hardware), and a tiny-scale ablation study (39 runs, 3-seed statistics) on the running ~ 2 –8M-parameter sanity-check network. The ablation study doubles as a methodological case: a striking single-run result – shared vs. separate attention projections – is shown, via three independent controls, to be a convergence-speed artifact rather than an architectural effect, reversing under the strictest control. No training run at target scale has been performed; the target-scale analysis is a compute-budget sizing exercise, and the ablation study is a memorization sanity check, not a generalization benchmark.

1 Introduction

1.1 Background

Small, edge-deployable language models (MobileLLM [Liu et al., 2024], SmolLM2 [Allal et al., 2025], Gemma-3 [Gemma Team, 2025]) are now routinely over-trained well past the Chinchilla-optimal token-to-parameter ratio [Hoffmann et al., 2022], on the premise that inference cost dominates total cost of ownership. That premise says nothing about training cost when the training hardware is *itself* constrained – the situation this report is concerned with. Three architectural levers target training-time efficiency in largely separate literatures: (i) sub-quadratic sequence mixing – local windowed attention [Beltagy et al., 2020, Jiang et al., 2023] and linear state-compressing mixers derived from state-space models (Mamba [Gu and Dao, 2023]) and the delta rule [Yang et al., 2024b,a]; (ii) sparse capacity scaling via MoE routing [Shazeer et al., 2017, Fedus et al., 2021, Dai et al., 2024, DeepSeek-AI, 2024]; (iii) non-autoregressive diffusion training [Austin et al., 2021, Lou et al., 2023, Sahoo et al., 2024, Nie et al., 2025], which generalizes masked language modeling [Devlin et al., 2018] into a full generative objective.

1.2 Problem statement

Each lever reports efficiency in its own currency – asymptotic sequence-length complexity, FLOPs-per-token, or generation parallelism – rarely all three, and rarely against the memory-bandwidth limits of the edge-class GPUs that motivate combining them. FLOP-centric MoE accounting implicitly assumes a bandwidth-favorable forward pass implies a bandwidth-favorable *training step*; Section 3.1 shows this fails once the backward pass is accounted for, because gradients flow through every expert reachable across a batch, not the subset any one token selected. **Research question:** for a hybrid linear-attention/MoE/diffusion model targeting edge-class training hardware, what governs achievable throughput – FLOPs or bandwidth – and which parameter count, active or total, is the correct lever to reduce?

1.3 Contributions

1. The Kairos architecture, specified directly from its implementation: PyramidalCodec (Algorithm 1) and KairosMemoryGate (Algorithm 2), for which we found no direct precedent in the literature; LiZAttention2 (Algorithm 3), Block-AttnRes (Algorithm 4), sparse MoE (§2.6), and the masked-diffusion curriculum (Algorithm 5), each adapted from a specific prior method.
2. A roofline-based compute analysis (§3.1) of a 200M-total/25M-active target design point, showing it is memory-bound and that its MoE backward pass scales with total, not active, parameters.
3. A tiny-scale ablation study (§3.2, 39 runs, 3-seed statistics) isolating the effect of codec design, attention sharing, depth, and MoE routing on the running sanity-check network.
4. A worked correlation-vs-causation case within that study (§3.2.1): a clear single-run effect that does not survive three independent controls, offered as a methodological cautionary example alongside the architecture.

1.4 Related work

Efficient attention. Full self-attention [Vaswani et al., 2017] scales quadratically in sequence length; the delta rule [Yang et al., 2024b,a] improves on plain linear attention with an error-correcting state update; Kairos runs it *bidirectionally* (forward pass plus a time-reversed pass, concatenated) rather than causally, matching the bidirectional context a masked-diffusion objective requires. Sliding-window attention [Beltagy et al., 2020, Jiang et al., 2023] is likewise run bidirectionally here, with rotary position embeddings [Su et al., 2021] and grouped-query attention [Ainslie et al., 2023]. Coupling a delta-rule branch and a window-attention branch through shared Q/K/V/O projections follows the TPTT construction [Furfaro, 2025].

Sparse MoE. MoE architectures [Shazeer et al., 2017, Fedus et al., 2021, Dai et al., 2024, DeepSeek-AI, 2024] decouple capacity from per-token compute. Kairos’s MoE layer subclasses Qwen2’s dense MLP [Qwen Team et al., 2024] for the non-routed path and DeepSeek-V3’s routed-expert layer [DeepSeek-AI, 2024] directly, rather than reimplementing either. The training-time memory-bandwidth cost of the router’s backward pass on bandwidth-starved hardware – central to §3.1 – is not, to our knowledge, separately quantified in this literature.

Cross-layer residual aggregation. Kairos’s Block-AttnRes replaces the fixed, unit-weight residual sum standard since ResNet [He et al., 2015]/Transformer stacks with the learned, block-windowed attention over prior layer outputs of Kimi Team [2026], used here as-is rather than re-derived (§2.5).

Diffusion-style language modeling. Kairos follows the MDLM [Sahoo et al., 2024] masking/reweighting formulation but adds a three-stage MAE $\rightarrow$ transition $\rightarrow$ diffusion curriculum (Algorithm 5), motivated by optimization instability observed when sampling the full masking distribution from a random initialization; the cited diffusion-LM literature typically does not stage this.

Scaling laws. Chinchilla [Hoffmann et al., 2022] is fitted on 70M–16B *dense* models; extrapolating to a 4M-active-parameter MoE model is out of range. Joint MoE scaling laws [Ludziejewski et al., 2025] instead parameterize loss in active parameters, tokens, and expert count, and report that the marginal value of tokens grows with expert count – relevant to §3.1’s token budget. A separate line analyzes the FLOPs-vs-sparsity trade-off directly [Abnar et al., 2025]; we build on it to isolate the total-vs-active distinction specifically in the backward pass.

2 Methods

2.1 Two configurations, not one

The codebase runs at two distinct scales that this report is careful not to conflate. The **PoC configuration** is the default in the project’s marimo notebook (`notebook/kairos_pretraining.py`): a ~ 14 – 15 M-parameter network used to sanity-check the pipeline via memorization (`overfit_test`) and per-modality loss checks. Its 7-expert, top-1 MoE is small; the notebook’s own comment claims top-1 routing converges too slowly for a fast sanity check, but §3.2’s controlled top-1-vs-top-2 comparison does not support this (0.281 vs. 0.272 min. loss) – the claim predates this report and should not be taken as settled. The **target configuration** is the 200M-total / 25M-active design point used in §3.1 purely to size a future training run; it has not been instantiated or trained. Table 1 gives both.

Figure 1 shows the shared data flow; both configurations instantiate the same modules, differing only in the sizes in Table 1.

2.2 PyramidalCodec: multi-scale byte encode/decode

Every modality is encoded to raw bytes/tokens, then to S parallel scales at patch sizes $p_s = \text{stride}^s$ ($s = 1 \dots S$; PoC: stride 3, $S = 4$, so $p = 3, 9, 27, 81$), fused, and decoded back the same way (Algorithm 1 shows `mode="conv"`, the PoC’s setting). A shared rank- r bottleneck mixer ($\text{Conv1d}_{d \rightarrow r} \rightarrow \text{GELU} \rightarrow \text{Conv1d}_{r \rightarrow d}$, cost $O(rd)$ against a dense 1×1 conv’s $O(d^2)$) supplies cross-channel context once before the per-scale grouped convolutions downsample; each scale gets its own such mixer again afterward. The grouped convolution’s group count is the one knob that trades compute for expressiveness: `channels_per_group=1` (default) is depthwise – cheapest, and the setting §3.2 shows is slowest to memorize; raising it toward d recovers a dense convolution’s expressiveness at proportionally higher cost. `mode="patch"` replaces the whole grouped-conv/mixer chain with a single $\text{Linear}(p_s d \rightarrow d)$ per scale, decoding through the *same* weight matrix transposed (tied encode/decode, no separate decoder parameters) – simpler, and empirically the fastest to memorize (§3.2), but without the grouped-convolution knob’s compute/expressiveness trade-off.

Decode mirrors this with grouped transpose convolutions (upsampling each scale back to L , trimmed to the original length before the pad) and a second set of per-scale mixers, then the same

Parameter	PoC (running)	Target (design point, untrained)
d_{model}	64	–
Attention heads (head dim)	4 (16)	–
Backbone layers	4	–
Codec scales / stride	4 / 3	–
AttnRes block size	4	–
Routed experts E / active k	7 / 1	32 / 4
Shared experts	1	–
FFN intermediate size	544	–
Backbone sharing across scales	yes ($\sim 75\%$ param saving)	–
Codec mode	<code>conv (channels_per_group=8)</code>	–
Cross-session memory bank	enabled	–
N_{total}	$\sim 14\text{--}15\text{M}$	200M
N_{act}	not yet logged (§3)	25M (4M for §3.1)
Target training tokens D	16 examples $\times$ 200 steps (overfit)	30B

Table 1: The two configurations this report distinguishes. Dashes mean the target design point has not been fixed at that granularity – only the parameter/token budget that drives §3.1 has been specified. N_{act} for the PoC config is computed by the same routine used at target scale (Eq. 1) but has not yet been logged from an instantiated model; it is listed as a pending measurement, not estimated here, to avoid quoting an unverified number. The PoC’s codec mode (`conv`) is chosen for training-time efficiency at real scale, not because it wins on the tiny memorization task in Table 6 – there, `patch` does; see §3.2 for why the two do not agree.

`concat/RMSNorm/fusion; mode="patch"`’s decode is the tied transpose of its encode `Linear` plus a learned bias, per scale.

2.3 KairosMemoryGate: cross-session recurrent-state memory

Kairos optionally carries DeltaNet recurrent state across sequences (`use_memory_gate=True`): before each new sequence, every scale’s every DeltaNet layer gets its SSM state seeded not from zero but from a learned blend of zero and a bank of prior states – either the immediately preceding batch’s own final state (`prev_cache`, always available once training has taken one step) or an externally supplied `memory_bank` (Algorithm 2). The blend is a single-head cross-attention over a rank- r' bottleneck ($r' = \max(8, \lceil \sqrt{d}/4 \rceil$) by default), not a fixed interpolation: the query is the (zero) current state, the keys/values are the bank’s states, so the gate can attend selectively to whichever prior state is most relevant rather than always averaging all of them equally.

2.4 LiZAttention2: shared-basis bidirectional hybrid attention

Each layer runs a bidirectional sliding-window branch (`KairosAttention`, window mask $|q - k| \leq w$ in both directions) and a bidirectional gated-DeltaNet branch (`KairosGatedDeltaNet`, run once forward and once on the time-reversed sequence, then flipped back and concatenated) over the *same* input; the two branches’ outputs are concatenated, normalized with RMSNorm [Zhang and Sennrich, 2019], and projected back to d (Algorithm 3). The DeltaNet branch’s Q/K/V/output projections are deleted and replaced by references to the SWA branch’s, so both mixers read the same linear projections of x ; only their mixing rule differs.

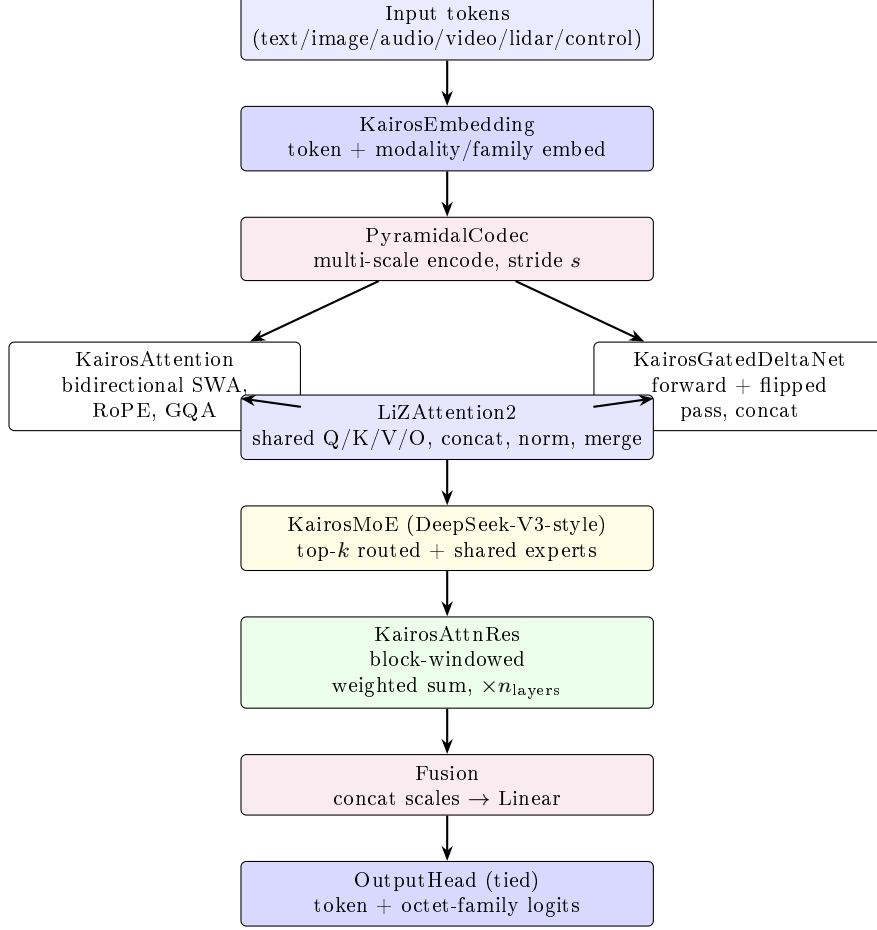


Figure 1: Kairos data flow ($\text{DiffusionBlock} \times \text{n_layers}$, wrapped by $\text{KairosDiffusionBackbone}$). Both attention branches see the same input and are bidirectional; the Block-AttnRes aggregator determines what each layer actually reads.

2.5 Block-AttnRes aggregation

Rather than a plain residual $x_{i+1} = x_i + f_i(x_i)$, Kairos follows the block-windowed Attention Residuals construction of Kimi Team [2026]: each layer’s *input* is a learned, softmax-weighted mix of the embedding, finalized block-sums of earlier layers, and a running partial sum of the current block (Algorithm 4); the layer’s own attention/FFN sublayers still use ordinary residual connections around h . With `attnres_block_size` = 1 this degenerates to a DenseNet-style [Huang et al., 2016] sum over all prior individual layer outputs; with block size = `n_layers` it degenerates to a single running sum (closer to a plain residual stream). The PoC configuration uses block size 4 over 4 layers, i.e. one block covering the whole network.

2.6 Sparse Mixture-of-Experts

KairosMoE subclasses DeepSeek-V3’s routed-expert layer [DeepSeek-AI, 2024] directly (only the expert-weight initialization is patched, since the upstream implementation leaves expert weights uninitialized); KairosFFN , used when MoE is disabled, subclasses Qwen2’s dense MLP [Qwen Team et al., 2024]. Each token is routed to k of E routed experts plus 1 or more always-on shared experts.

Algorithm 1 PyramidalCodec encode (`mode="conv"`)

Require: sequence $x \in \mathbb{R}^{B \times L \times d}$, patch sizes $p_1, \dots, p_S$

- 1: $h \leftarrow \text{BottleneckMixer}_{\text{pre}}(x^\top)$ $\triangleright$ shared cross-channel context, cost $O(rd)$
 - 2: **for** $s = 1$ to S **do**
 - 3: $h_s \leftarrow \text{PadToMultiple}(h, p_s)$
 - 4: $h_s \leftarrow \text{GroupedConv1d}(h_s; \text{kernel} = p_s, \text{stride} = p_s, \text{groups} = d/\text{channels_per_group})$
 - 5: $\text{scale}_s \leftarrow \text{BottleneckMixer}_s(h_s)$
 - 6: **end for**
 - 7: $h \leftarrow \text{concat}(\text{scale}_1, \dots, \text{scale}_S)$ $\triangleright Sd$ -wide
 - 8: $h \leftarrow \text{RMSNorm}(h)$
 - 9: **return** $\text{Linear}_{Sd \rightarrow d}(h), \{\text{scale}_1, \dots, \text{scale}_S\}$ $\triangleright$ fused repr. for the backbone; scales kept for decode
-

Algorithm 2 Memory-bank gating, applied per (scale, DeltaNet layer)

Require: zero state $s_0 \in \mathbb{R}^{B \times D}$, memory bank $M \in \mathbb{R}^{M \times D}$ (stacked prior states)

- 1: **if** M is empty **then**
 - 2: **return** s_0 $\triangleright$ no-op: identical to never having enabled the gate
 - 3: **end if**
 - 4: **if** M has exactly one row **then**
 - 5: **return** that row, broadcast to batch size B $\triangleright$ degenerate case: nothing to attend over
 - 6: **end if**
 - 7: $q \leftarrow \text{Linear}_{D \rightarrow r'}(s_0); K, V \leftarrow \text{Linear}_{D \rightarrow r'}(M)$
 - 8: $\text{out} \leftarrow \text{MultiheadAttention}_{h=1}(q, K, V)$
 - 9: **return** $\text{Linear}_{r' \rightarrow D}(\text{out})$
-

The PoC configuration ($E = 7, k = 1$) is under-sparse relative to the target ($E = 32, k = 4$); see §3.2 for why this was chosen and why the choice does not hold up under test.

The active-parameter count used throughout this report is computed directly, not estimated after the fact:

$$N_{\text{act}} = N_{\text{total}} - N_{\text{expert}} + N_{\text{expert}} \cdot \frac{k}{E}, \quad (1)$$

where N_{expert} sums only parameters under modules whose name contains `".experts."` – shared experts, the router gate, the attention/DeltaNet path, and the codec are treated as fully dense (every token activates them), matching how `count_active_parameters()` walks the module tree in the codebase.

2.7 Masked-diffusion training objective

Given a clean sequence x_0 , each row samples $p \sim U(\epsilon, p_{\text{max}})$ and a Bernoulli(p) mask over non-prompt, non-pad positions; masked positions are replaced with random tokens (and, where present, random octet-family ids) to form x_t . The model predicts x_0 at masked positions from x_t ; the loss is cross-entropy, optionally reweighted by a continuous blend factor $\alpha \in [0, 1]$ between plain CE ($\alpha = 0$) and full $1/p$ reweighting ($\alpha = 1$), following the MDLM formulation [Sahoo et al., 2024]. Self-conditioning [Chen et al., 2022] (probability 0.5 in the current config) occasionally feeds a detached, no-grad warm-up prediction back in as extra context, matching how `generate()` uses it at inference time.

The stage schedule ($p_{\text{max}}^{(t)}, \alpha^{(t)}$) is a pure function of the global step (`stage_mask_schedule`):

Algorithm 3 LiZAttention2 forward (one layer)

Require: hidden states $x \in \mathbb{R}^{B \times L \times d}$, RoPE table, attention mask

- 1: $q, k, v \leftarrow \text{swa.q_proj}(x), \text{swa.k_proj}(x), \text{swa.v_proj}(x)$ $\triangleright$ shared with DeltaNet
 - 2: $\text{swa_out} \leftarrow \text{BidirWindowAttn}(q, k, v, \text{window } w, \text{RoPE})$
 - 3: $\text{delta_out}_f \leftarrow \text{GatedDeltaRule}(q, k, v)$
 - 4: $\text{delta_out}_b \leftarrow \text{flip}(\text{GatedDeltaRule}(\text{flip}(q, k, v)))$
 - 5: $\text{delta_out} \leftarrow \text{swa.out_proj}(\text{delta_out}_f + \text{delta_out}_b \text{ combination})$
 - 6: $\text{out} \leftarrow \text{concat}(\text{swa_out}, \text{delta_out})$ $\triangleright 2d$ -wide
 - 7: $\text{out} \leftarrow \text{RMSNorm}(\text{out})$
 - 8: **return** $\text{Linear}_{2d \rightarrow d}(\text{out})$
-

Algorithm 4 Block-AttnRes backbone forward

Require: embedding x_0 , layers $f_1, \dots, f_L$, block size S , learned $w \in \mathbb{R}^d$ per aggregator

- 1: $\text{completed} \leftarrow []$; $\text{partial} \leftarrow \text{None}$; $\text{count} \leftarrow 0$
 - 2: **for** $i = 1, \dots, L$ **do**
 - 3: $\text{sources} \leftarrow [x_0] \cup \text{completed} \cup (\{\text{partial}\} \text{ if } \text{partial} \neq \text{None})$
 - 4: $V \leftarrow \text{stack}(\text{sources})$; $K \leftarrow \text{RMSNorm}(V)$
 - 5: $\text{logits} \leftarrow w^\top K$ (over the stacked/source dimension)
 - 6: $h \leftarrow \sum_{\text{sources}} \text{softmax}(\text{logits}) \cdot V$
 - 7: $x_i \leftarrow f_i(h)$ $\triangleright$ ordinary residual attention + FFN inside f_i
 - 8: $\text{partial} \leftarrow x_i$ if partial is None else $\text{partial} + x_i$; $\text{count} += 1$
 - 9: **if** $\text{count} = S$ **then**
 - 10: $\text{completed.append}(\text{partial})$; $\text{partial} \leftarrow \text{None}$; $\text{count} \leftarrow 0$
 - 11: **end if**
 - 12: **end for**
 - 13: **return** $\text{RMSNorm}(x_L)$
-

flat at ($p_{\max}^{\text{MAE}} = 0.3$, $\alpha = 0$) for the MAE stage, linearly ramped to ($p_{\max} = 1.0$, $\alpha = 1$) over the transition stage, then flat at the diffusion target – chosen so that resuming from a checkpoint depends only on the persisted step count, not on which pipeline call produced it.

3 Results

Two unrelated kinds of result follow, at two different scales: §3.1 is an analytical sizing exercise for the 200M/25M **target** design point that has not been trained; §3.2 is real, measured data (39 runs) from the ~ 2 –8M-parameter **PoC** that has. Neither substitutes for the other, and Table 6’s codec/attention findings should not be assumed to hold at the target’s scale.

3.1 Compute budget analysis (target design point)

This section applies only to the 200M/25M **target** design point of Table 1; it is a sizing exercise for a training run that has not been executed, not a measurement of the running PoC.

Algorithm 5 Masked-diffusion training step (`compute_loss`)

Require: batch x_0 , prompt lengths, pad mask, stage schedule $(p_{\max}^{(t)}, \alpha^{(t)})$ at step t

- 1: $p \leftarrow (p_{\max}^{(t)} - \epsilon) \cdot U(0, 1)^B + \epsilon$ $\triangleright$ one p per row
 - 2: $\text{mask} \leftarrow \text{Bernoulli}(p)$ restricted to non-prompt, non-pad positions
 - 3: $x_t \leftarrow x_0$ with $x_t[\text{mask}] \leftarrow$ random tokens
 - 4: $\text{logits} \leftarrow \text{model}(x_t, \text{logits_mask} = \text{mask})$ $\triangleright$ project only masked positions
 - 5: $\ell \leftarrow \text{CE}(\text{logits}, x_0[\text{mask}]); \quad \ell \leftarrow \ell \cdot (1 + \alpha^{(t)}(1/p - 1))$ if $\alpha^{(t)} > 0$
 - 6: **return** $\text{mean}(\ell)$ $\triangleright$ + octet-family CE if family ids present
-

3.1.1 Total training FLOPs

For a dense transformer, $C \approx 6ND$ [Kaplan et al., 2020]; for MoE, compute scales with active, not total, parameters [Ludziejewski et al., 2025]:

$$C_{\text{Kairos}} = 6 \times 4\text{M} \times 30\text{B} = 7.2 \times 10^{17} \text{ FLOPs},$$

roughly 720 $\times$ cheaper in raw FLOPs than an estimated dense Gemma-3 270M [Gemma Team, 2025] run ($\sim 5 \times 10^{20}$ FLOPs). This comparison alone overstates the win, as the roofline analysis below shows.

3.1.2 Roofline model

Achievable throughput is $\min(\pi, \mathcal{I} \cdot b_s)$, with ridge point $\mathcal{I}^* = \pi/b_s$ [Williams et al., 2009] (Table 2, Figure 2). Kairos’s training step has $\mathcal{I}_{\text{train}} = 6N_{\text{act}}/4N_{\text{act}} = 1.5 \text{ FLOP/byte}$ – independent of model size, deep in the memory-bound region on every GPU class considered.

GPU	π (TFLOPS)	b_s (TB/s)	Ridge point $\mathcal{I}^*$ (FLOP/byte)
A100	312	2.0	156
H100 SXM5	989	3.35	295

Table 2: Peak compute, HBM bandwidth, ridge points.

3.1.3 The MoE bandwidth bottleneck

Pass	Bytes/token	Driving factor
Forward	$4N_{\text{act}} = 16\text{MB}$	only active experts + attention read
Backward	$4N_{\text{total}} = 800\text{MB}$	gradients flow through <i>all</i> experts via router
Adam	$8N_{\text{total}}/B$	amortized; negligible for $B \gtrsim 32$

Table 3: Bytes moved per token by pass, target design point.

The backward pass reads N_{total} , not N_{act} , because the router’s gradient depends on every expert’s contribution to the batch loss [Abnar et al., 2025]. As batch size grows,

$$B_{\text{eff}} = 4N_{\text{total}} + \frac{8N_{\text{total}}}{B} \xrightarrow{B \rightarrow \infty} 4N_{\text{total}}.$$

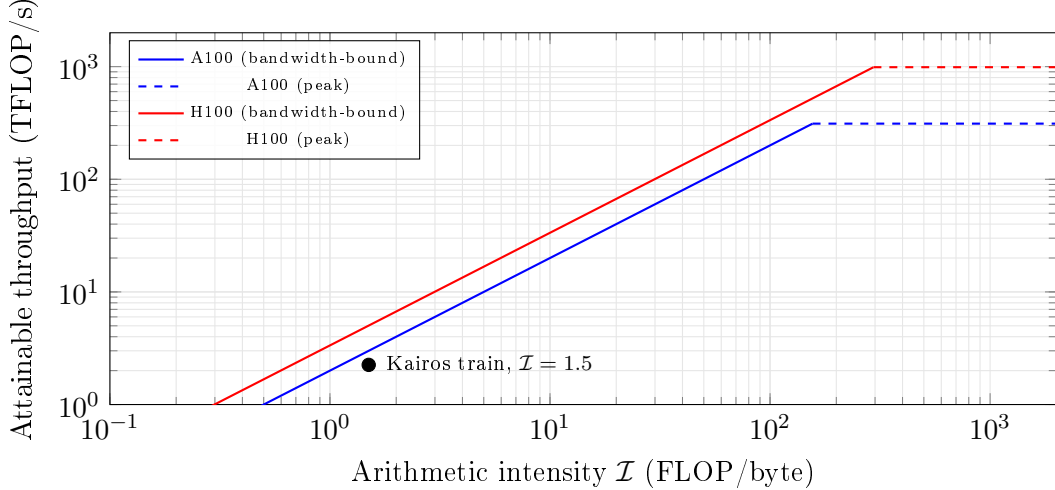


Figure 2: Kairos’s training arithmetic intensity vs. A100/H100 ridge points – deep in the memory-bound regime on both.

The backward-pass bandwidth cost of a MoE model is governed by its *total*, not active, parameter count – capping the achievable training-time speedup from sparsity even though it is fully realized at inference time.

3.1.4 Throughput and wall-clock estimates

GPU	b_s (GB/s)	tok/s (theory)	tok/s (MFU 25%)	Days for 30B tokens
T4	300	375	~ 94	$\sim 3\,700$
RTX 5060Ti	600	750	~ 187	$\sim 1\,850$
A100 40G	2\,000	2\,500	~ 625	~ 555
H100 SXM5	3\,350	4\,188	$\sim 1\,047$	~ 332

Table 4: Theoretical-upper-bound throughput at $N_{\text{total}} = 200\text{M}$. Not measured: no run has reached this scale. See `docs/experiments/planned_experiments.md` (item 12) for the instrumented benchmark that would replace this table with real numbers.

None of the four classes reach a practical wall-clock budget at the current N_{total} ; T4/RTX 5060Ti – the stated low-compute target – are orders of magnitude over any reasonable budget. The fastest lever is reducing N_{total} (e.g. to $\sim 50\text{M}$) at fixed N_{act} , a $4\times$ cut in backward-pass bytes, before any kernel-level work.

3.1.5 Scaling-law context

Chinchilla [Hoffmann et al., 2022]’s fit is out of range for a 4M-active-parameter model; joint MoE scaling laws [Ludziejewski et al., 2025] instead predict the marginal value of tokens *grows* with expert count, softening the concern. Table 5 places the target token ratio ($D/N_{\text{act}} = 7,500\times$) against small dense-model precedents – none of which use sparse routing, so they bound plausibility of coherent output at this active-parameter scale but say nothing about the bandwidth bottleneck above.

Model	N_{total}	N_{act}	Tokens	D/N	Notes
Pythia-14M [Biderman et al., 2023]	14M	14M	$\sim 300\text{B}$	$\sim 21,000\times$	dense, $6\text{L}\times\text{d128}$
Pythia-31M [Biderman et al., 2023]	31M	31M	$\sim 300\text{B}$	$\sim 9,700\times$	dense, $6\text{L}\times\text{d256}$
TinyStories-1M [Eldan and Li, 2023]	$\sim 1\text{M}$	$\sim 1\text{M}$	$\sim 3\text{B}$	$\sim 3,000\times$	domain-scoped
TinyStories-33M [Eldan and Li, 2023]	$\sim 33\text{M}$	$\sim 33\text{M}$	$\sim 3\text{B}$	$\sim 90\times$	domain-scoped
Kairos PoC (running)	$\sim 14\text{--}15\text{M}$	TBD	overfit-scale	n/a	MoE, sanity check only
Kairos target (design)	200M	4M	30B	$7,500\times$	MoE, multimodal, untrained

Table 5: Small-model precedents next to both Kairos configurations.

3.2 Tiny-scale ablation study

The PoC network ($d_{\text{model}} = 64$, MoE off unless noted) was used to run 39 controlled sanity-check experiments via `kairos overfit (kairos==0.4.1.dev46)`, memorizing 16 repeated sentences under a fixed mask regime ($p_{\text{max}} = 0.3$, no reweighting) for 600 steps unless noted. Every config is versioned under `configs/ablations/`; full results, raw per-step curves, and single-run vs. 3-seed methodology notes are in `docs/experiments/` in the repository. This is a memorization sanity check, not a generalization benchmark; effect directions should not be assumed to transfer to real, diverse data without re-testing (§3.2.1 shows exactly this can fail).

Configuration	Min. loss (mean $\pm$ std, 3 seeds)	Params
<code>liz2</code> attention, depthwise codec (<code>channels_per_group=1</code>)	0.354 ± 0.193	1.98M
<code>vanilla</code> (SWA-only) attention, depthwise codec	0.707 ± 0.103	1.81M
<code>conv</code> codec, <code>channels_per_group=8</code> (PoC’s actual setting)	0.075 ± 0.031	2.50M
<code>conv</code> codec, dense (<code>channels_per_group=64</code>)	0.028 ± 0.007	8.10M
<code>patch</code> codec (dense <code>nn.Linear</code> per scale)	0.018 ± 0.002	4.93M

Table 6: Codec and attention choice under 3-seed statistics. `channels_per_group=1` (architectural default when unspecified) trades memorization speed for a $2.7\times$ parameter saving over `patch`; the PoC’s actual setting (`channels_per_group=8`, Table 1) recovers most of that gap at under half `patch`’s cost. `liz2` beats `vanilla` on every individual seed despite overlapping mean $\pm$ std ranges – but, like §3.2.1’s shared-vs-separate result, this gap is a convergence-speed effect, not a capacity one: at 3 000 steps `vanilla` alone reaches 0.0014, matching `liz2`’s 0.0011 (Table 7).

A single-run sweep (one seed; read as indicative, not exact – Table 6’s spread sets the noise floor) covered six further factors: network depth (1–8 layers, non-monotonic, no benefit past 1 layer at this scale), MoE routing (top-1 vs. top-2: 0.281 vs. 0.272, indistinguishable – the notebook’s claim that top-1 is "slow to converge" is not supported here), the attention window (`attnres_block_size`: no effect below `n_layers`, saturates at or above it), codec scale count (monotonic: 4 scales beat 3 beat 2), sequence length (monotonic: longer context helps), and batch size (monotonic: larger batches converge faster in step count, as expected when they approach full-batch on 16 examples). Full tables are in `docs/experiments/ablations_codec_attention.md`.

3.2.1 A correlation-vs-causation case: shared vs. separate QKV

LiZAttention2’s shared Q/K/V/O projections (§2.4) initially looked like a clear architectural win: with sharing disabled (independent projections per branch, otherwise identical), min. loss rose from 0.227–0.369 to 0.610–0.858 across repeated 600-step runs. Three independent controls were run

before accepting this as an architectural effect, rather than a run-specific artifact of a too-easy, too-short setup:

Control	Shared	Separate	Outcome
More steps (3 000 vs. 600)	0.0011	0.0011	gap closes
Diverse data (63 distinct sentences)	2.269	2.285	gap closes
QKV frozen after step 100	0.705	0.521	gap reverses

Table 7: Three controls on the shared-vs-separate-QKV effect (docs/experiments/liz2_shared_vs_separate_followup_raw.jsonl holds the full per-step curves). Sharing’s apparent advantage was faster early convergence – shared projections receive a gradient signal from both attention branches every step, an effective higher early learning rate – not higher achievable capacity: given enough steps or diverse data both variants reach the same floor, and removing the extra gradient signal (freezing QKV after step 100) reverses the ranking.

The lesson generalizes beyond this one comparison: a single-run, single-config result on a tiny memorization task is a hypothesis, not a finding, until checked against a step-budget control, a data-scale control, and (where a mechanism is suspected) a control that isolates it directly.

4 Discussion

For the target design point, bandwidth – not FLOPs – governs achievable training throughput, and N_{total} , not N_{act} , is the lever that sets it, because N_{total} drives the backward pass’s byte traffic. MoE efficiency claims reported on data-center hardware, where bandwidth is generous relative to N_{total} , do not transfer cleanly to bandwidth-starved edge hardware for this reason. Separately, the PoC’s under-sparse routing (§2.6) should be re-examined given §3.2’s top-1-vs-top-2 result, rather than left as originally chosen.

4.1 Limitations

The target-scale analysis (§3.1) is a compute-budget sizing exercise for a run that has not been performed: its throughput figures are theoretical upper bounds, the Chinchilla extrapolation is out of its fitted range, and the roofline model ignores kernel launch overhead, activation recomputation, and multi-GPU communication, all of which would lower real throughput further. The ablation study (§3.2) is a memorization sanity check on 16–63 repeated or near-duplicate sentences at $d_{\text{model}} = 64$; only the codec/attention comparison in Table 6 and the QKV-sharing case (§3.2.1) carry 3-seed or multi-control statistics – the other six factors are single-run and should be read as indicative. The memory gate and 8-layer depth point were both compute-budget-limited to inconclusive step counts and are flagged as open, not resolved.

5 Conclusion

Kairos is currently a ~ 2 –8M-parameter proof of concept exercising a hybrid bidirectional DeltaNet/SWA, MoE, masked-diffusion architecture end-to-end, with a separate 200M/25M design point used only to size a future run. The code-derived algorithms (3–5), the roofline analysis, and the 39-run ablation study together give two falsifiable next steps: reduce N_{total} at fixed N_{act} before scaling up, and re-run the single-run ablation factors (§3.2) under the same 3-seed protocol

as Table 6 before citing them individually. The QKV-sharing case (§3.2.1) is offered as a standing reminder, for this project and others, that a single striking result on an easy proxy task is a hypothesis until it survives a step-budget, a data-scale, and a mechanism-isolating control.

References

- Samira Abnar, Harshay Shah, Dan Busbridge, Alaaeldin Mohamed Elnouby Ali, Josh Susskind, and Vimal Thilak. Parameters vs flops: Scaling laws for optimal sparsity for mixture-of-experts language models. *arXiv preprint arXiv:2501.12370*, 2025.
- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, Thomas Wolf, et al. Smollm2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. *arXiv preprint arXiv:2107.03006*, 2021.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. *arXiv preprint arXiv:2304.01373*, 2023.
- Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. *arXiv preprint arXiv:2208.04202*, 2022.
- Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- Fabien Furfaro. Tptt: Transforming pretrained transformer into titans. *arXiv preprint arXiv:2506.17671*, 2025.
- Gemma Team. Gemma 3 technical report. Google DeepMind, 2025.

- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. *arXiv preprint arXiv:1608.06993*, 2016.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Kimi Team. Attention residuals, 2026.
- Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*, 2024.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- Jan Ludziejewski, Maciej Pi  ro, Jakub Krajewski, Maciej Stefaniak, Micha   Krutul, Jan Ma  snicki, Marek Cygan, Piotr Sankowski, Kamil Adamczewski, Piotr Mi  s, and Sebastian Jaszczur. Joint moe scaling laws: Mixture of experts can be memory efficient. *arXiv preprint arXiv:2502.05172*, 2025.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Qwen Team, An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.

- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. In *Communications of the ACM*, volume 52, pages 65–76. ACM, 2009.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024a.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024b.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019.